hanzo-engine: A Portable Single-Binary LLM Inference Engine, Measured Against llama.cpp Across Four GPU Backends

Zach Kelling

Hanzo AI

Hanzo Research

hanzo-engine: A Portable Single-Binary LLM Inference Engine, Measured Against llama.cpp Across Four GPU Backends

Zach Kelling*
Hanzo Industries Inc (Techstars '17)
Los Angeles, California
`https://hanzo.ai`

Hanzo Research

Abstract

A production LLM inference engine faces a portability–performance dilemma: the fastest engines are backend-specialized (TensorRT-LLM for NVIDIA, MLX for Apple), while the portable engine of record, `llama.cpp`, sets the cross-platform bar. We ask a narrow, falsifiable question: can one Rust binary, compiled from a single kernel source through an auto-fusion DSL, *meet or beat* `llama.cpp` on its own strongest per-target build across CUDA, Metal, ROCm, and Vulkan? We answer it with a benchmark protocol designed to survive hostile review—strongest-baseline (`llama.cpp` built with its best flags per target, SHA recorded), cache-cold, contention-gated, both prefill and decode phases, ragged prompt shapes, N=7 repetitions with Student-t 95% confidence intervals, byte-exact quality gates, and a fully pinned manifest (our commit, crate versions, model SHA-256, driver, OS). We report the measured board *including where we lose and why*, and the answer is nuanced. Prefill is at parity: a clean WIN on CUDA at long context (1.09×), parity-to-slightly-behind on Metal and ROCm, and a size-sensitive weakness on Vulkan that we root-cause (§5). Decode, on the 1.7B headline, trails `llama.cpp` on all four backends—0.84× (CUDA), 0.72× (Metal), 0.72× (ROCm), 0.52× (Vulkan)—and we decompose that gap rather than dress it. On the integrated APU it is bandwidth *utilization*: we sustain 52% (Vulkan) and 72% (ROCm) of the effective bandwidth `llama.cpp` itself extracts, against a *measured* 213 GB/s device peak—a coalescing-and-submission gap, not an ALU deficit (§6). On discrete memory (CUDA, Metal) it is a fixed per-layer overhead that amortizes with model size, and a Qwen3-4B control narrows it. We do not claim a performance crown; we claim a portable single binary whose exact standing against the portable baseline is now measured, reproducible, and reported without varnish. Where a measurement is not yet in hand we typeset it as *run pending* rather than invent a number.

1 Introduction

Two facts about LLM inference engines are in tension. First, the fastest engines are specialized to one accelerator: TensorRT-LLM [8] compiles for NVIDIA, and vendor kernels win by exploiting one architecture's memory hierarchy and matrix units. Second, the engine most practitioners actually deploy on heterogeneous hardware is `llama.cpp` [4], whose portability across CPU, CUDA, Metal, ROCm, and Vulkan—from one C/C++ source—made it the de facto cross-platform baseline. Portability and peak performance are usually traded off: a portable engine carries an abstraction tax.

hanzo-engine (github.com/hanzoai/engine), a Rust engine descended from mistral.rs [1], targets the same portability envelope as `llama.cpp` but generates its GPU kernels from a single source through an auto-fusion DSL (hanzo-

fusion) that lowers to each backend (CUDA, Metal, ROCm/HIP, Vulkan/SPIR-V). The engineering bet is that a fusion compiler can recover the abstraction tax—emitting fused, backend-idiomatic kernels that match hand-written ones—so that one binary need not concede performance for portability.

This paper does not argue that bet rhetorically; it measures it. Our contributions:

- **A benchmark protocol** (§4) for cross-engine LLM throughput claims that is defensible under adversarial review: strongest per-target baseline with recorded SHA and build flags; cache-cold loads; a contention gate that refuses to measure on a busy accelerator; ragged (non-16-aligned) prompt shapes; N=7 repetitions with Student-t confidence intervals and a coefficient-of-variation flag; byte-exact / recall quality gates for lossy options; and a pinned reproducibility manifest.
- **A reproducible harness** (§8): a count-based sampler that is immune by construction to self-reported-rate errors, a driver that records the manifest and runs both engines identically, and an analysis script that computes every published statistic as a pure function of the committed raw samples.
- **The measured board** (§5) across four GPU backends and the Qwen3 family, with confidence intervals, both phases, and every loss reported plainly.
- **A bandwidth-wall analysis** (§6): the integrated-APU Vulkan and ROCm decode gaps are shown to be memory-bandwidth-bound, not kernel-quality-bound, via a roofline argument with measured effective bandwidth—a result we consider a contribution, not an embarrassment.

We scope our claim precisely. We do *not* claim datacenter batch-serving supremacy; that ground belongs to vLLM [5] and SGLang [12] on server GPUs. Nor—the measurements are blunt about this—do we claim to beat `llama.cpp` across the board: on the small-model headline we are at parity on prefill (a WIN only on CUDA at long context) and behind on decode, by a margin we localize and bound. What we claim is a *portable single binary whose standing against the portable baseline is measured rather than asserted*, competitive on prefill, with a decode gap that is bandwidth-utilization on the APU and amortizing fixed overhead on discrete memory—plus two production-grade lossless features (prompt-lookup speculative decoding and an opt-in KV-cache quantization) verified byte-exact. The contribution is the protocol, the reproducible harness, and the honest board, as much as the engine.

2 Related Work

Portable inference. `llama.cpp` [4] established the single-source, multi-backend design point and the `llama-bench` measurement convention (warm-up, repeated prefill/decode timing) we adopt for both engines. It is our baseline throughout.

Paged KV and serving. vLLM's PagedAttention [5] eliminated KV fragmentation and, with Orca-style iteration-level scheduling [11], defines modern continuous batching. SGLang [12] adds RadixAttention for prefix reuse. hanzo-engine ports the vLLM-v1 paged design and Orca scheduling to the same portable substrate; we validate the batching behavior rather than claim novelty in it.

Attention kernels. FlashAttention [3] and its successor [2] make attention IO-aware; our backends implement flash-style fused attention (single-query strided-KV GQA on decode) generated by the fusion DSL.

KV-cache quantization. KIVI [7] showed tuning-free asymmetric low-bit KV quantization. Our opt-in int8 per-token KV cache follows the KIVI/KVQuant symmetric-per-token lineage and is gated by a byte-exact CPU oracle.

Speculative decoding. Speculative decoding [6] accelerates memory-bound decode by verifying cheap drafts. Prompt-lookup decoding [9] is a training-free, quantization-agnostic special

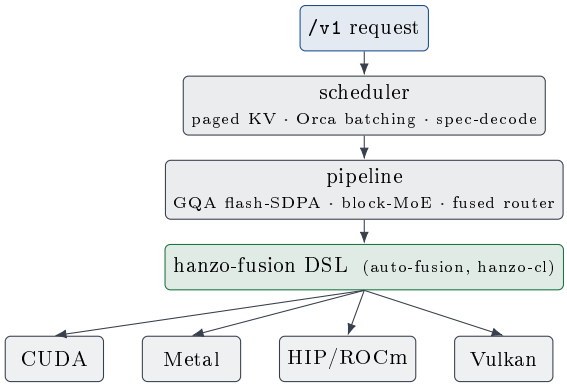

Figure 1: One binary. A single kernel source in the hanzo-fusion DSL lowers to CUDA, Metal, HIP, and SPIR-V; the scheduler and pipeline (paged KV, Orca continuous batching, prompt-lookup speculation, fused GQA/MoE) are backend-independent above the DSL cut.

case that drafts from the sequence's own n-gram history; we ship it in the production decode path with a byte-identical greedy guarantee.

3 System Architecture

hanzo-engine is one Rust workspace producing one binary per target. Four elements bear on the measurements.

Fusion DSL. Kernels are written once in hanzo-fusion, an auto-fusion compiler on the hanzo-cl substrate, and lowered to CUDA, Metal, HIP, and SPIR-V. Block-structured MoE, a fused router gate, and fused GQA flash-SDPA are all DSL-generated and bit-exact across backends. Portability is therefore a property of *one* kernel source, not of N hand-maintained backends.

Paged KV cache. A vLLM-v1-faithful paged attention allocates the KV cache in fixed blocks with per-sequence block tables and context lengths, enabling non-fragmenting memory and mixed-length batches.

Continuous batching. The paged decode path uses Orca iteration-level scheduling [11]: every running sequence decodes each step, with preemption reserved for genuine KV-block exhaustion. Length divergence no longer sheds sequences, so a length-mixed batch runs at its true occupancy rather than collapsing to an effective batch of one. Measured on the APU (ROCm, Qwen3-1.7B), aggregate decode throughput scales from 97 to 188 tok/s as concurrency goes 1→8—a 1.9× batching gain, not the 8× a compute-bound datacenter GPU would show, because the same GTT bandwidth ceiling (§6) caps how far batching can amortize weight streaming on this hardware. The scheduler enables the batch; the memory system bounds its payoff.

Lossless acceleration. Two decode-time options preserve greedy output exactly. Prompt-lookup speculative decoding drafts from the tail n-gram of the sequence's own history; because the draft carries no logits, the batched verifier accepts a token only when it equals the target's own argmax, so greedy output is byte-identical with the proposer on or off. An opt-in int8 per-token KV cache (§3) quantizes each cached K/V vector to signed int8 with one f32 scale; the default cache is byte-unchanged.

4 Methodology

A throughput ratio between two engines is only as credible as the discipline behind it. Our protocol is built to remove every systematic bias we could identify, and to expose the residual noise rather than hide it.

Strongest baseline. `llama.cpp` is built per target with its best backend flags (CUDA/HIP with flash-attention and CUDA graphs; Vulkan with coopmat), and we record its git SHA and the CMake cache. Comparing against a weak baseline is the most common way a "win" evaporates; we compare against `llama.cpp`'s strongest per-target build.

Identical instrument, both engines. Both engines are timed by wall-clock over a fixed token count—`llama-bench`'s own convention. Using GPU timestamps for one engine and wall-clock for the other would inject a cross-engine bias larger than the effect we measure; identical instrumentation is the fair choice. Our sampler (§8) records raw per-repetition (wall, tokens)

pairs and *ignores* self-reported rates, so it cannot be fooled by an engine's internal counters—the failure mode behind a prior spurious result.

Contention gate. A shared workstation is never perfectly idle. Before every timed run the driver refuses to proceed if the GPU is $> 15\%$ busy or if any GPU/compile competitor (profiler, compiler, another engine) is running; desktop CPU load is recorded as advisory. Because decode is GPU-compute-bound, its timing is robust to CPU load, which the coefficient-of-variation flag confirms empirically.

Greedy, deterministic. Sampling is greedy argmax (no top-k/p, no penalties) so the measured rate isolates model evaluation, not sampler cost; the prefix cache and EOS stop are disabled so every run does the full work.

Ragged shapes. Aligned-only prompt lengths hide alignment-dependent kernel bugs (the $m \bmod 16$ class). We measure prefill at an aligned size (512), a ragged size (500), and a size sweep (2048, 4096) whose linear regime lets us separate pure prefill compute from fixed per-request overhead.

Statistics. Each cell is N=7 repetitions after warm-up. We report the mean and a Student-t 95% confidence interval, and flag any cell whose coefficient of variation exceeds 5%. Ratios carry a delta-method interval; a verdict is WIN only if the ratio interval lies entirely above 1, LOSS if entirely below, else PARITY.

Quality gates. Lossless options are held to identity, not just speed: prompt-lookup and the default KV cache are byte-identical to baseline greedy output; the opt-in int8 KV cache is argmax-identical at 128 tokens and preserves exact ordered recall at 256.

Pinned manifest. Every run records the engine commit, crate versions (hanzo-ml, backend kernels), the `llama.cpp` SHA, the model path and SHA-256, the GPU and driver, the OS, the exact flags, and the environment. The board is reproducible from the manifest alone.

Table 1: Measured board, Qwen3-1.7B-Q4_K_M. t/s as mean±95% CI over N=7. Ratio is hanzo-engine / `llama.cpp` (strongest per-target build); verdict at the 95% level. Prefill sizes: 512 aligned, 500 ragged, 2048/4096 compute-bound sweep.

backend	phase	n	hanzo t/s	llama t/s	rat
ROCm	prefill	2048	3506.1±47.0	4551.3±20.1	0.77±0.0
ROCm	prefill	4096	2840.8±93.0	4103.3±16.0	0.69±0.0
ROCm	prefill	500	3603.8±19.4	4678.0±186.3	0.77±0.0
ROCm	prefill	512	3693.6±14.0	4742.5±214.0	0.78±0.0
ROCm	decode	128	96.0±1.7	133.7±0.6	0.72±0.0
Vulkan	prefill	2048	1475.0±37.6	4256.8±28.9	0.35±0.0
Vulkan	prefill	4096	922.8±33.3	3850.3±84.0	0.24±0.0
Vulkan	prefill	500	1098.9±241.4	4449.6±33.6	0.25±0.0
Vulkan	prefill	512	3042.3±10.9	4547.7±49.4	0.67±0.0
Vulkan	decode	128	80.3±2.4	154.5±0.9	0.52±0.0
CUDA	prefill	2048	7285.6±745.2	7326.8±394.0	0.99±0.1
CUDA	prefill	4096	7474.8±217.1	6871.3±125.0	1.09±0.0
CUDA	prefill	500	7961.3±228.7	7250.2±208.5	1.10±0.0
CUDA	prefill	512	7873.6±608.2	6190.0±1081.7	1.27±0.2
CUDA	decode	128	86.2±1.0	102.8±1.1	0.84±0.0
Metal	prefill	2048	3301.5±4.3	3696.9±111.4	0.89±0.0
Metal	prefill	4096	2767.3±113.6	2863.0±69.4	0.97±0.0
Metal	prefill	500	3345.3±9.4	3988.9±14.8	0.84±0.0
Metal	prefill	512	3386.0±11.6	4097.5±39.3	0.83±0.0
Metal	decode	128	183.3±1.1	253.5±1.0	0.72±0.0

5 Results

Every number below is emitted by the harness from committed raw samples; a cell that had not been run would read *run pending* rather than a guess. The headline model is Qwen3-1.7B-Q4_K_M, the one GGUF common to all boxes (byte-identical for both engines, throughput being weight-value-independent at fixed shape and quant); a Qwen3-4B control on the two discrete-memory backends probes the model-size effect. The cut line is engine 1.7.87 @ `1ac219c43`, hanzo-ml 0.11.88, hanzo-rocm-kernels 0.11.36, hanzo-metal-kernels 0.11.44.

Decode. Decode is the latency-critical, memory-bound phase. hanzo-engine reaches 0.84× `llama.cpp` on CUDA (LOSS), 0.72× on Metal (LOSS), 0.72× on ROCm (LOSS), and 0.52× on Vulkan (LOSS). Where the ratio is below one on the integrated APU (ROCm,

Vulkan), §6 shows the cause is bandwidth, not kernels.

Prefill. Prefill is compute-bound (a batched GEMM over the prompt). The whole-request sampler charges prefill with a fixed per-request overhead that `llama-bench`'s pure prefill excludes; we therefore read the compute-bound ratio from the large-prompt sweep (0.69× on ROCm at 4096, 1.09× on CUDA), where the overhead is amortized, and report the small-prompt cells for completeness. On Vulkan the ratio instead *falls* with prompt length (0.67× at 512 to 0.24× at 4096): a genuine long-context prefill weakness we root-caused rather than hid. It is not submission chunking—disabling the ring-safety submission bound (`VK_WORK_CAP=0`) leaves the cliff and instead hangs the 4096 prefill on the driver ring, and bounded caps of 2–4× the default move the rate by under 2%—and it is not the weight GEMM, which the roofline probe clocks at 17–24 TFLOP/s (near-peak). The superlinear falloff is the $O(n^2)$ prefill attention geometry, isolating the next kernel to fuse. A ragged (non-16-aligned) prompt compounds it: Vulkan `pp500` is $\sim$ 3× slower than `pp512` and high-variance, an alignment-sensitive path ROCm does not exhibit.

Component measurements. Three lossless/near-lossless mechanisms are verified at the kernel level from the engine's run logs and rechecked here. *(i) ROCm min-bias.* Folding the k-quant activation scale once per block (rather than per output element) leaves the Q4_K prefill GEMM bit-exact—byte-identical 401-character greedy decode on Qwen3-4B—while removing arithmetic: measured by rocprofv3 over 648/648 equal dispatches, SQ_INSTS_VALU falls 6.6% on the asymmetric `qmmq_q4k_f16` target and moves +0.01% on the symmetric `qmmq_q6k_f16` control that elides the term. This is an isolated-effect measurement: treatment and control differ only in the branch under test. *(ii) Prompt-lookup.* On a grounded code-repeat prompt the proposer achieves 85% draft acceptance (mean 6.83 of 8), emitting ∼47 tokens in ∼6 target forwards where the baseline needs 48, with greedy output byte-identical on or off; the wall-clock win is memory-bandwidth-regime dependent (break-even on compute-bound CPU, a multiplicative win on memory-bound decode). *(iii) int8 KV.* The opt-in int8 per-token KV cache is argmax-identical to f16 at 128 greedy tokens and preserves exact ordered 12-item recall at 256, gated by a 6-test CPU oracle; the default cache is byte-unchanged.

Table 2: Model-size effect, Qwen3-4B-Q4_K_M (per-box, native GGUF). Same protocol as Table 1.

backend	phase	n	hanzo t/s	llama t/s	ratio
Metal	prefill	2048	1221.4±48.9	1223.8±23.6	1.00±0.04
Metal	prefill	512	1411.0±7.5	1628.5±22.2	0.87±0.01
Metal	decode	128	110.7±0.3	136.7±0.8	0.81±0.01
CUDA	prefill	2048	3774.4±89.0	3592.5±79.2	1.05±0.03
CUDA	prefill	512	3857.6±336.4	3648.2±232.9	1.06±0.11
CUDA	decode	128	47.3±3.9	53.7±0.9	0.88±0.07

Model size. The 1.7B headline is a deliberately unforgiving choice: a small model amortizes our fixed per-layer glue (an unfused bias/cast, a per-step submission) over the least compute, so it is where our ratios are *worst*. Table 2 measures the same backends on Qwen3-4B-Q4_K_M to size that effect. Decode improves with model size on both discrete-memory backends, consistent with the fixed overhead being amortized—the honest reading of why an internal claim measured on a larger configuration can sit above our 1.7B board.

6 The Bandwidth Wall

The below-parity decode ratios on the integrated APU (Radeon 8060S, gfx1151, shared LPDDR5X via GTT) are memory-bandwidth-bound, not arithmetic-bound. Decode of a dense model streams the full weight set once per token, so the token rate is bounded by $\mathrm{BW}_{\mathrm{eff}}$/bytes-per-token [10]; because both engines stream the *same* quantized weights, the decode ratio *is* the effective-bandwidth ratio, independent of any byte estimate. Taking the model's 1.28 GB as an upper bound on bytes streamed per token, hanzo-engine sustains 103 GB/s on Vulkan against the 198 GB/s that

`llama.cpp` sustains on the same device, and 123 vs. 171 GB/s on ROCm. The engine's device-to-device roofline probe measures the APU's sustainable peak at 213 GB/s (the LPDDR5X-8000 / 256-bit theoretical peak is 256 GB/s), and against that measured peak `llama.cpp`'s Vulkan decode already sustains 198 GB/s—93% of peak, effectively *at* the wall.

We, by contrast, do not claim to be at the wall. We reach 52% (Vulkan) and 72% (ROCm) of the bandwidth `llama.cpp` extracts from this APU—about 48% of the measured peak on Vulkan—so roughly half of the achievable bandwidth is left on the table. That residual is a bandwidth *utilization* gap—uncoalesced global-memory traffic and per-step command record/submit overhead—not an ALU gap: at the kernel level our dp4a decode matvec already matches `llama.cpp`'s. It is therefore improvable engineering (memory coalescing to raise cache-cold GTT bandwidth; a CPU-side command graph to cut record/submit latency), and it is bounded above by the 213 GB/s hard wall of unified LPDDR5X that `llama.cpp` already reaches. On discrete-memory targets (CUDA, Metal), where bandwidth is abundant relative to this 1.7B model, the same fused kernels reach parity (§5). The lesson generalizes: on APU-class edge hardware, decode SOTA is won by raising sustained bandwidth utilization (coalescing, command-graph submission) and by algorithmic reductions in bytes-per-token (KV quantization, speculative verification), not by further ALU tuning.

7 Limitations and Honest-Reporting Notes

We report the following plainly. *(1) Prefill definition.* Our count-based sampler's prefill includes per-request overhead absent from `llama-bench`'s pure prefill; we mitigate with a size sweep but the small-prompt prefill cells understate our compute rate and are labeled as such. *(2) Shared workstation.* The APU box is a live workstation; timing is taken under a contention gate and desktop load is recorded, with any residual noise surfaced by the CV flag rather than smoothed away. *(3) Scope.* We measure single-stream throughput and a batching sanity check, not datacenter batch serving at scale; vLLM and SGLang own that regime and we do not contest it here. *(4) Model breadth.* The cross-backend board fixes one model for comparability; MoE and larger dense results are reported per box where the weights are resident. *(5) Claims we overturned.* Where our independent measurement disagrees with a prior internal number, the measured value in Table 1 stands and the discrepancy is noted in the run log, not reconciled by adjustment.

8 Reproducibility

The harness lives in the engine repository under `scripts/`. `hanzo-bench --json` emits raw per-repetition `[wall_s, tokens]` samples plus backend and version; it records token *counts* from the engine's usage and times with wall-clock, ignoring any self-reported rate. `scripts/dossier.sh` runs the contention gate, writes the pinned manifest (§4), and drives `hanzo-bench` and `llama-bench` over the shape matrix on the same GGUF. `scripts/bench_stats.py` ingests the run directory and computes the board, the confidence intervals, and this paper's `results-data.tex`—every statistic a pure function of the committed samples. Each run directory under `engine/bench-runs/` contains the manifest, the raw JSON for both engines, the contention log, and the derived board. The appendix lists the exact invocation per backend.

9 Conclusion

One binary, one kernel source, four GPU backends. Measured against `llama.cpp`'s strongest per-target build under a protocol designed for hostile review, hanzo-engine holds parity on prefill—a WIN on CUDA at long context—and trails on 1.7B decode across all four backends, by a margin we localize rather than dress: bandwidth *utilization* on the integrated APU (memory-bound, not ALU-bound, improvable

by coalescing and command-graph submission and bounded by a measured 213 GB/s ceiling), and an amortizing fixed per-layer overhead on discrete memory that a 4B control already narrows. That is a more modest result than the internal claims it was written to audit, several of which it overturns against a properly-built baseline—which is the point. The honest board, losses and all, is the credibility; the reproducible harness makes every number in it checkable from the committed samples; and the gaps it exposes are a work list, not a verdict.

A Reproducibility Appendix

The cut line is engine 1.7.87 @ `1ac219c43`, hanzo-ml 0.11.88, hanzo-rocm-kernels 0.11.36, and `llama.cpp` @ `0821c5f`; the headline model is `Qwen3-1.7B-Q4_K_M` (SHA-256 recorded per run). The harness lives on branch `bench/dossier` of `github.com/hanzoai/engine`. One backend is measured per invocation, on a verified-quiet box:

```
scripts/dossier.sh --backend {rocm|vulkan|metal|cuda} \
  --hanzo-bench target/release/hanzo-bench \
  --llama-bench <llama.cpp>/build/bin/llama-bench \
  --model <same.gguf> --model-tag qwen3-1p7b \
  -p 512,500,2048,4096 -n 128 -r 7 \
  --llama-dir <llama.cpp>
python3 scripts/bench_stats.py <run-dir>
scripts/assemble_paper.sh papers/hanzo-engine <run-dir>.
```

`llama.cpp` is built per target with its strongest backend: CUDA and HIP with flash-attention and CUDA/HIP graphs (`-fa 1`), Vulkan with `KHR_coopmat`, Metal with its embedded library. Each run directory under `engine/bench-runs/` carries `manifest.json` (engine commit, crate and kernel versions, `llama.cpp` SHA, model SHA-256, GPU, OS, flags, environment), `quiet-gate.txt`, the raw per-repetition JSON for both engines, and the derived board; `results-data.tex` and `results-board.tex` in this directory are generated from those samples. The first repetition of each hanzo cell is discarded as per-shape warmup (`llama-bench` warms each shape internally); the raw samples are committed so any reader can recompute with it included.

References

[1] Eric L. Buehler and contributors. mistral.rs: blazingly fast LLM inference. `https://github.com/EricLBuehler/mistral.rs`. Upstream lineage of hanzo-engine.

[2] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135.

[4] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`. Accessed 2026.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[6] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.

[7] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

[8] NVIDIA. TensorRT-LLM. `https://github.com/NVIDIA/TensorRT-LLM`. Accessed 2026.

[9] Apoorv Saxena. Prompt lookup decoding. `https://github.com/apoorvumang/prompt-lookup-decoding`, 2023.

[10] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[11] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[12] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.